

บทที่ 4

การทดลองและผลการทดลอง

ในบทนี้จะกล่าวถึงแบบจำลองและผลที่ได้จากการทดลอง ซึ่งแสดงความสามารถในการจัดการใช้งานเครือข่ายสู่ภายนอก โดยส่งผลกระทบต่อการทำงานของโปรแกรมบิตทอเรนต์น้อยที่สุด โดยทดสอบดูประสิทธิภาพการทำงานผ่านระบบเครือข่ายกับการทำงานแบบปกติของโปรแกรมบิตทอเรนต์ รวมถึงการลดจำนวนการรับส่งข้อมูลที่เกิดจากการโหลด นอกจากนี้ยังทดสอบการทำงานว่าเป็นไปตามที่ได้ออกแบบไว้หรือไม่ ปัญหาที่พบและการแก้ไขได้บันทึกไว้ที่ช่วงท้ายของบทนี้

4.1 แบบจำลองและคุณสมบัติของอุปกรณ์ที่ใช้ทดลอง

เนื่องจากข้อจำกัดด้านทรัพยากร ความแตกต่างของประสิทธิภาพเครื่อง ในการทดสอบนี้แบบจำลองใช้เครื่องคอมพิวเตอร์ทั้งหมดสี่เครื่องตั้งอยู่บนเครือข่ายเดียวกัน และอีกหนึ่งเครื่องอยู่บนละเครือข่าย โดยมีหน้าที่แตกต่างกันไป ซึ่งก็มี เครื่องที่ทำหน้าที่เป็นแคชคอยโหลดงาน Torrent เมื่อเครื่องภายในเครือข่ายเรียกไฟล์มาใช้ จะตั้งเป็นพร็อกซีของเครื่องภายในเนื่องจากอยู่ในเครือข่ายเดียวกัน เครื่องภายในทั้งสามที่ทำหน้าที่โหลด โดยทั้งสามเครื่องนี้เป็นเครื่องในเครือข่ายเครือข่ายหนึ่ง ที่ติดต่อผ่านเครื่องแคชเวลามีการโหลดไฟล์ที่เป็น Metadata และเครื่องที่อยู่ภายนอกเครือข่าย ทำหน้าที่ปล่อยไฟล์ให้กับเครือข่าย ทำงานเป็นแทรคเกอร์ของงานแลกเปลี่ยนไฟล์

- คุณสมบัติของเครื่องที่ทำงานเป็นแคช
 - Central Processing Unit: Intel® Pentium® 4 CPU 3.2 GHz
 - Main board: ASUS® P4P800 Special Edition
 - Memory: 1024 MB Dual Channel at 800 MHz
 - Operating System: Fedora Core 4 Linux, Kernel 2.6
 - Network Adapter: SMC EZ Card 10/100 PCI (SMC1211 series), connected at 100.0 Mbps
 - IP Address: 161.246.7.191
 - BitTorrent Client: BitTorrent 4.4
 - Web Server: Lampp Web Service pack for Linux. Included: Apache, MySql and PhPMyAdmin

- คุณสมบัติของเครื่องที่ทำงานเป็นเครื่องภายนอก
 - Central Processing Unit: AMD Athlon™ XP 2200+ 1.8 GHz
 - Main board: ASUS® A7V8X-X
 - Memory: 768 MB
 - Operating System: Microsoft® Windows® XP Professional, Service Pack 2
 - Network Adapter: VIA Rhine 2 Fast Ethernet Adapter, connected at 100.0 Mbps
 - IP Address: 161.246.5.120
 - BitTorrent Client: BitComet 0.59]
 - Web Server: Xampp Web Service pack for Windows. Included: Apache, MySQL and PhpMyAdmin

- ไฟล์ที่ใช้ทดสอบ
 - ไฟล์ขนาด 11.3 MB โดยมี Metadata เป็น test10.torrent ขนาด 7.30 KB ตั้งให้ดาวน์โหลดไว้ที่ <http://161.246.5.120/test10.torrent>
 - ไฟล์ขนาด 53.04 MB โดยมี Metadata เป็น test50.torrent ขนาด 8.59 KB ตั้งให้ดาวน์โหลดไว้ที่ <http://161.246.5.120/test50.torrent>
 - ไฟล์ขนาด 92.48 MB โดยมี Metadata เป็น test100.torrent ขนาด 14.7 KB ตั้งให้ดาวน์โหลดไว้ที่ <http://161.246.5.120/test100.torrent>
 - ไฟล์ขนาด 200.27 MB โดยมี Metadata เป็น test200.torrent ขนาด 15 KB ตั้งให้ดาวน์โหลดไว้ที่ <http://161.246.5.120/test200.torrent>
 - ไฟล์ขนาด 386.01 MB โดยมี Metadata เป็น test400.torrent ขนาด 31 KB ตั้งให้ดาวน์โหลดไว้ที่ <http://161.246.5.120/test400.torrent>

ไฟล์ทุกไฟล์เปิดงาน Torrent โดยใช้เครื่องที่อยู่ภายนอก ทำงานภายใต้ BitComet 0.59 เครื่องที่อยู่เครื่องภายนอก ทำหน้าที่เป็น Tracker, Seeder และผู้ให้ Metadata files ไปด้วย เครื่องแคช ทำหน้าที่เป็น Gateway, Tracker, Seeder, Peer และผู้ให้ Metadata files เมื่อมีการ Redirect ไปด้วย เครื่องโหลด ทำหน้าที่เป็น Peer และผู้โหลดไฟล์ Metadata ทั้งโดยตรงและผ่านพร็อกซี่

4.2 การตั้งค่าต่างๆในการทดลอง

- เครื่องภายนอก

สร้าง Metadata ของแต่ละไฟล์ขึ้นมาตามที่กำหนด โดยใช้โปรแกรม BitComet สร้างไฟล์แบบรับแต่ Peers จากในแทรคเกอร์เท่านั้น กำหนด URL ไปที่ไอพีแอดเดรสของเครื่องภายนอก ทำการเก็บไฟล์ Metadata ไว้ที่โฟลเดอร์ htdocs ของโปรแกรม Xampp เพื่อให้สามารถเข้าถึงได้จากอินเทอร์เน็ต เมื่อเริ่ม Apache Web Service จากนั้น เอา Metadata ของแต่ละไฟล์ไปลงทะเบียนเข้ากับแทรคเกอร์

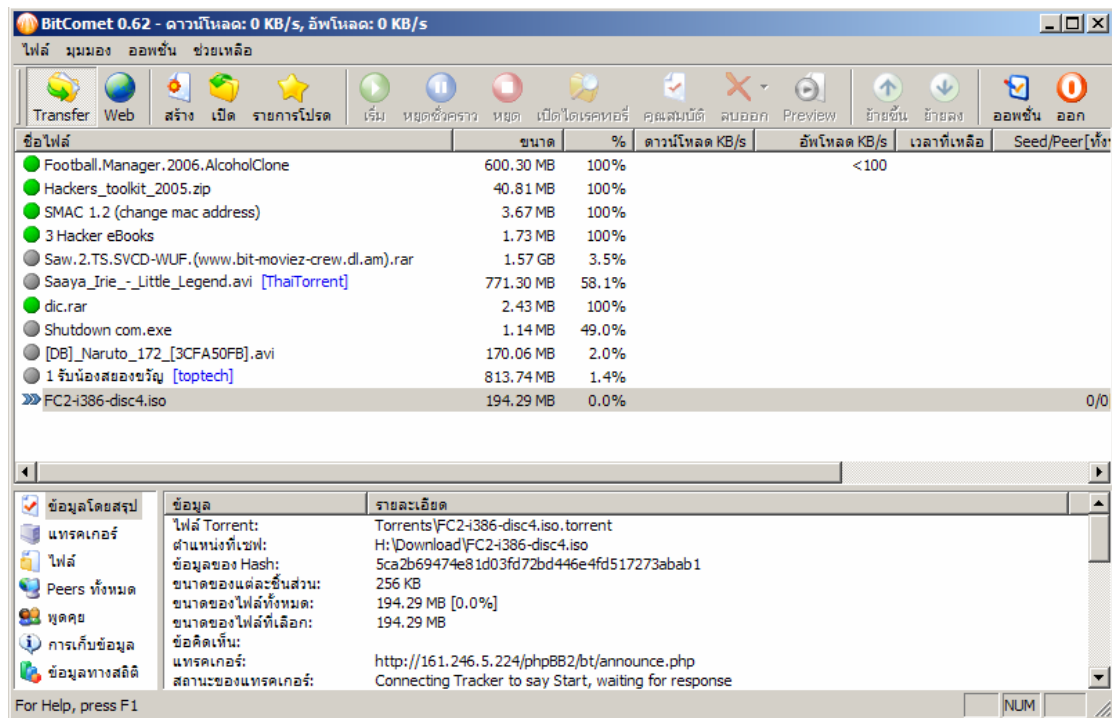
แทรคเกอร์ที่ใช้คือ Torrent Pier บน phpBB ซึ่งต้องติดตั้งเมื่อเริ่ม Apache Web Service และ MySQL กับ PhpMyAdmin แล้ว โดยติดตั้ง phpBB ก่อน เพื่อความสะดวกในการทดสอบ ทำการติดตั้ง phpBB และ Torrent Pier ที่โฟลเดอร์ htdocs เช่นกัน การลงทะเบียนกับแทรคเกอร์ที่ง่ายที่สุด คือการ Attach file ที่ Web Board Download ของ Torrent Pier ซึ่งจะมีการยืนยันการลงทะเบียนเข้ากับแทรคเกอร์โดยสมบูรณ์ ให้ลงทะเบียนให้ครบทั้งสามไฟล์ ทั้งนี้สามารถตรวจสอบการลงทะเบียนกับแทรคเกอร์ได้ โดยเรียก PhpMyAdmin ขึ้นมาเข้าไปดูฐานข้อมูลของ Torrent Pier ที่ตาราง Torrentfile หรือ bt_torrent

จากนั้นให้เรียก Metadata ทั้งสามไฟล์มาทำงานในโปรแกรม BitComet เพื่อทำการปล่อยไฟล์ให้กับเครือข่ายเมื่อมีการติดต่อเข้ามา ทั้งนี้ เพื่อป้องกันการติดต่อกับเครื่องภายในผ่าน UDP ให้ทำการปิด DHT Function ของโปรแกรม BitComet ออก และตั้งค่า Upload speed ให้สูงสุดไว้เพื่อการทดสอบ

ปิด Firewall ของเครื่องออกเพื่อให้แทรคเกอร์สามารถติดต่อกับเครื่องอื่นๆได้ และการส่งชิ้นส่วนไฟล์ใน Bittorrent มีพอร์ตที่ไม่แน่นอน จึงเป็นการยากที่จะทำ Allow Access

- เครื่องภายใน

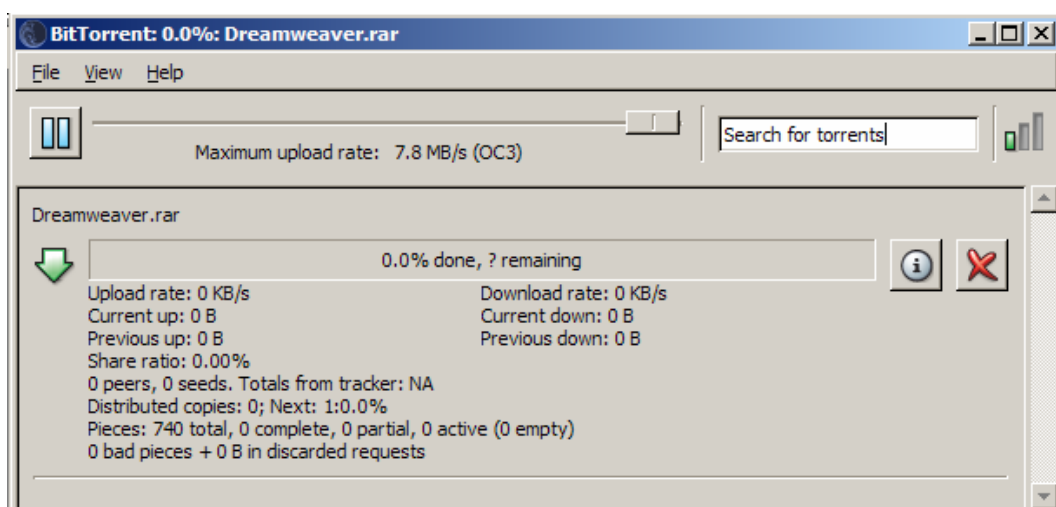
ทำการโหลดไฟล์ Metadata จากเครื่องภายนอกไว้ก่อน เพื่อใช้ทดสอบการทำงานแบบไม่ผ่านแคช จากนั้น ตั้ง Gateway ของเครื่องให้ผ่านแคช เพื่อเปิดระบบแคชสำหรับการทดลองอื่นๆ ปิดระบบ DHT function ของ BitComet ออกเพื่อป้องกันการโหลดจากแหล่งอื่นนอกเหนือจากการทดลอง



รูปที่ 4.1 User Interface ของ BitComet

- เครื่องแกช

เปิด Web Service ของ Lammup ขึ้นมา นำไฟล์ที่ต้องใช้ในการ redirect แก้ไขและโหลดไฟล์ metadata ไปติดตั้งไว้ที่โฟลเดอร์ httdocs ของโปรแกรมเพื่อเรียกใช้ได้สะดวกเมื่อ SquidGaurd redirect มาให้ ทำการเตรียมแทรคเกอร์ให้พร้อมโดยเปิด MySQL ให้พร้อมให้บริการ ทำการลบข้อมูลไฟล์ที่เคยโหลดไว้ก่อนออกจากแกช ซึ่งก็คือโฟลเดอร์ Original, Modified และ Return ใน httdocs/torrent แล้วเรียก BitTorrent ที่ได้ปรับแต่งไว้แล้วมาพร้อมทำงาน ติดตั้งโปรแกรม Ntop สำหรับตรวจจับการเชื่อมต่อไว้



รูปที่ 4.2 User Interface ของ BitTorrent 4.4.0

4.3 การทดลองและผลการทดลอง

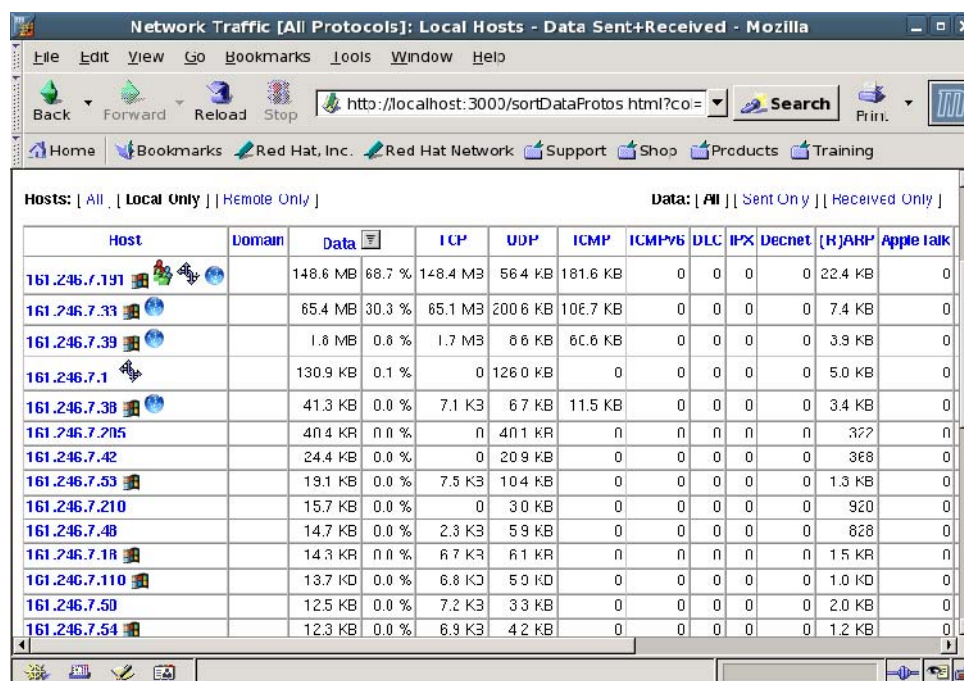
4.3.1 การทดลองที่ 1 ประสิทธิภาพการลดจำนวนการรับส่งข้อมูลที่ติดต่อออกสู่ภายนอก

ทำการทดสอบว่า แบบจำลองที่ใช้สามารถลดการติดต่อออกภายนอกเครือข่ายได้แค่ไหน โดยเปรียบเทียบระหว่างการติดต่อโดยตรงและการติดต่อผ่านแคช

เนื่องจากจำนวนของเครื่องและจำนวนของไฟล์ที่ทำงานพร้อมกันมีผลต่อประสิทธิภาพของระบบพอๆกับ ขนาดของไฟล์ จึงแยกเป็นกรณีต่างๆ มาเปรียบเทียบความสามารถของระบบ

การทดสอบนี้ ใช้โปรแกรม Ntop ซึ่งมีความสามารถในการดูจำนวนข้อมูลและการรับส่งข้อมูลต่างๆ ของเครื่องภายในเครือข่ายที่ติดต่อผ่านอุปกรณ์ของเครื่องที่มีโปรแกรมนี้ โดยติดตั้งโปรแกรมไว้ที่เครื่องแคชที่อยู่กับ Gateway จึงสามารถตรวจสอบการรับส่งข้อมูลของเครื่องทั้งสามได้ โปรแกรม Ntop นี้ใช้ WinPCap เป็นตัวจับข้อมูล รายละเอียดของโปรแกรมนี้ สามารถดูได้ที่

<http://www.ntop.org/>



Host	Domain	Data	TCP	UDP	ICMP	ICMPv6	DLC	IPX	Decnet	HTTP	AppleTalk
161.246.7.191		148.6 MB 68.7 %	148.4 M3	56.4 KB	181.6 KB	0	0	0	0	22.4 KB	0
161.246.7.33		65.4 MB 30.3 %	65.1 M3	200.6 KB	106.7 KB	0	0	0	0	7.4 KB	0
161.246.7.39		1.8 MB 0.8 %	1.7 M3	8.6 KB	60.6 KB	0	0	0	0	3.9 KB	0
161.246.7.1		130.9 KB 0.1 %	0	126.0 KB	0	0	0	0	0	5.0 KB	0
161.246.7.38		41.3 KB 0.0 %	7.1 K3	6.7 KB	11.5 KB	0	0	0	0	3.4 KB	0
161.246.7.205		40.4 KB 0.0 %	0	40.1 KB	0	0	0	0	0	3.2 KB	0
161.246.7.42		24.4 KB 0.0 %	0	20.9 KB	0	0	0	0	0	3.6 KB	0
161.246.7.53		19.1 KB 0.0 %	7.5 K3	10.4 KB	0	0	0	0	0	1.3 KB	0
161.246.7.210		15.7 KB 0.0 %	0	3.0 KB	0	0	0	0	0	9.2 KB	0
161.246.7.48		14.7 KB 0.0 %	2.3 K3	5.9 KB	0	0	0	0	0	8.2 KB	0
161.246.7.18		14.3 KB 0.0 %	6.7 K3	6.1 KB	0	0	0	0	0	1.5 KB	0
161.246.7.110		13.7 KB 0.0 %	6.8 K3	5.9 KB	0	0	0	0	0	1.0 KB	0
161.246.7.50		12.5 KB 0.0 %	7.2 K3	3.3 KB	0	0	0	0	0	2.0 KB	0
161.246.7.54		12.3 KB 0.0 %	6.9 K3	4.2 KB	0	0	0	0	0	1.2 KB	0

รูปที่ 4.3 หน้าจอการดูจำนวนการเชื่อมต่อของโปรแกรม Ntop

ทุกครั้งก่อนการทดสอบทุกกรณี จะมีการลบข้อมูลเก่าและเตรียมโปรแกรม Ntop ใหม่ โดยทำการลบงานที่โหลดออกจากโปรแกรม BitComet และหยุดการทำงานของ Ntop เพื่อลบสถิติที่มีอยู่แล้ว ก่อนการทดสอบกรณีที่โหลดพร้อมกับแคช จะทำการลบข้อมูลออกจากแทรคเกอร์และ BitTorrent และไฟล์ Metadata ที่อยู่ในเครื่อง

การทดสอบที่ 1.1 ดูขนาดการรับส่งข้อมูลของเครื่องสองเครื่อง ที่ทำงานผ่านระบบแคช ด้วยไฟล์ขนาดต่างๆ

ทำการเรียก Ntop ให้ทำงานก่อนเริ่มการโหลดข้อมูล ใช้เครื่องสองเครื่องในเครือข่าย โหลดไฟล์ metadata ผ่านแคชเพื่อเรียกให้แคชโหลดไปด้วย ระหว่างที่โหลด ให้ยุติการใช้งานเครือข่ายทั้งหมดเพื่อให้ได้ผลที่ใกล้เคียงกับการใช้งานที่เกิดจากการโหลดของ BitTorrent อย่างเดียว เก็บผลต่างๆเมื่อโหลดเสร็จ

ผลการทดสอบ

ตารางที่ 4.1 ขนาดของการรับส่งข้อมูลที่เกิดจากการติดต่อออกนอกเครือข่าย ภายใต้ระบบแคช

ขนาดไฟล์	ขนาดข้อมูลส่งออก	ขนาดข้อมูลรับเข้า
11.3 MB	218.2 KB	12.3 MB
53.04 MB	739.8 KB	56.6 MB
92.48 MB	2.1 MB	96.8 MB

สังเกตได้ว่า จำนวนข้อมูลขาออกที่เกิดขึ้น เป็นส่วนที่เกิดขึ้นจากการขอชิ้นส่วนจากเครื่องภายนอก และการติดต่อกับ Tracker ส่วนข้อมูลขาเข้า เป็นชิ้นส่วนไฟล์ที่ทำการโหลดมาจากเครื่องภายนอก ขนาดการโหลดที่เกิดขึ้นนั้น ใหญ่กว่าขนาดของไฟล์เล็กน้อย เพราะมีการติดต่อจาก Tracker และการติดต่อที่เกิดจาก Protocol ของเครือข่ายเอง

การทดสอบที่ 1.2 ดูขนาดการรับส่งข้อมูลของเครื่องสองเครื่อง ที่ทำงานตามการโหลดปกติ ด้วยไฟล์ขนาดต่างๆ

ลบข้อมูลเก่าในเครื่องทั้งสามเครื่อง เริ่มโปรแกรม Ntop เมื่อพร้อมที่จะทดสอบ โหลดไฟล์จากเครื่องภายนอกโดยตรงเพื่อทำงานแบบไม่ผ่านแคช แล้วเก็บข้อมูลการใช้งานเครือข่ายเมื่อโหลดเสร็จ

ผลการทดสอบ

ตารางที่ 4.2 ขนาดของการรับส่งข้อมูลที่เกิดจากการติดต่อออกนอกเครือข่าย โดยการทำงานปกติ

ขนาดไฟล์	ขนาดข้อมูลส่งออก	ขนาดข้อมูลรับเข้า
11.3 MB	558.4 KB	24.7 MB
53.04 MB	1.7 MB	114.2 MB
92.48 MB	4.9 MB	192.6 MB

จากตารางที่ 4.1 และ 4.2 เห็นได้ชัดว่า ความแตกต่างที่เกิดจากการโหลดในแบบไม่ใช่แคช กับใช้ระบบแคช คือจำนวนการใช้งานเครือข่ายทั้งขาเข้าและขาออก มีขนาดประมาณสองเท่าของ ขนาดไฟล์ที่โหลดจริง ซึ่งมาจากการโหลดไฟล์ของเครื่องทั้งสองเครื่องแยกกัน ไม่เหมือนในตอน ที่หนึ่งที่เครื่องแคชเครื่องเดียวติดต่อออกภายนอก ซึ่งขนาดการรับส่งไฟล์ มากกว่าขนาดของไฟล์ จริงเพียงเล็กน้อย

การทดสอบที่ 1.3 ดูประสิทธิภาพการลดขนาดการใช้งานเครือข่ายออกภายนอกของระบบแคช เมื่อ จำนวนเครื่องเพิ่มขึ้น

เพื่อดูว่า ประสิทธิภาพการลดการใช้งานเครือข่าย จะสามารถทำงานได้ดีแม้ว่าจะมีจำนวน เครื่องที่โหลดไฟล์หนึ่งหนึ่งมากขึ้น จึงให้เพิ่มจำนวนเครื่องที่ทำงานในเครือข่ายภายใต้ระบบแคช แล้วเก็บข้อมูลมาเปรียบเทียบ โดยทดลองที่สามเครื่อง และสี่เครื่อง

การทดลอง ลบค่าเก่าในเครื่องแคชออกเพื่อดูการโหลดไฟล์พร้อมกับเครื่องอื่นๆ ทำการ เริ่ม Ntop เมื่อพร้อม แล้วให้ทั้งสามเครื่อง โหลดไฟล์ metadata ของไฟล์ขนาดประมาณ 100 MB ผ่านระบบแคช แล้วเริ่มทำงาน เก็บข้อมูลเมื่อการโหลดเสร็จสิ้น

ทำซ้ำอีกรอบโดยเพิ่มจำนวนเครื่องเป็นสี่เครื่อง

นำผลการโหลดไฟล์ขนาด 100 MB จากการทดลองในตอนหนึ่งมาเปรียบเทียบ

ผลการทดสอบ

ตารางที่ 4.3 ขนาดของการรับส่งข้อมูลที่เกิดจากการติดต่อออกนอกเครือข่าย เปรียบเทียบจำนวน เครื่อง โดยใช้ไฟล์ขนาด 100 MB ผ่านระบบแคช

จำนวนเครื่อง	ขนาดข้อมูลส่งออก	ขนาดข้อมูลรับเข้า
2	2.1 MB	96.8 MB
3	2.0 MB	96.2 MB
4	2.2 MB	92.9 MB

การทดสอบที่ 1.4 ดูประสิทธิภาพการใช้งานเครือข่ายออกภายนอกแบบปกติ เมื่อจำนวนเครื่อง เพิ่มขึ้น

ทำการทดลองเหมือนกับการทดสอบที่ 1.3 แต่ให้ใช้การโหลดโดยตรงจากเครื่องภายนอก เพื่อไม่ต้องทำงานผ่านระบบแคช แล้วนำผลการโหลดไฟล์ขนาด 100 MB จากการทดลองในตอน ที่ 2 มาเปรียบเทียบ

ผลการทดสอบ

ตารางที่ 4.4 ขนาดของการรับส่งข้อมูลที่เกิดจากการติดต่อออกนอกเครือข่าย เปรียบเทียบจำนวนเครื่อง โดยใช้ไฟล์ขนาด 92.48 MB โดยการทำงานแบบปกติ

จำนวนเครื่อง	ขนาดข้อมูลส่งออก	ขนาดข้อมูลรับเข้า
2	4.9 MB	192.6 MB
3	6.7 MB	286.3 MB
4	9.3 MB	354.2 MB

จากตารางที่ 4.3 และ 4.4 สังเกตได้ว่า จำนวนเครื่องนั้น แทบจะไม่ส่งผลกระทบต่อการใช้งานเครือข่ายของ BitTorrent ภายใต้การทำงานของแคชเลข ที่เป็นเช่นนี้ก็เพราะเครื่อง Internal Peer ของระบบจะเป็นเครื่องเดียวที่ติดต่อออกนอกเครือข่าย ทำให้ขนาดการส่งและรับข้อมูล มากกว่าขนาดของไฟล์เพียงเล็กน้อย (จากความคลาดเคลื่อนที่มาจาก Protocol ต่างๆของเครือข่าย) ซึ่งในการทำงานแบบปกติของ BitTorrent ขนาดของการใช้งานเครือข่ายโดยเฉพาะที่ข้อมูลเข้า มีขนาดใกล้เคียงกับผลคูณของขนาดของไฟล์ที่โหลดกับจำนวนเครื่อง ซึ่งแสดงให้เห็นอย่างชัดเจนว่าระบบแคชสามารถลดการใช้งานเครือข่ายได้มากขนาดไหนเมื่อนำมาเปรียบเทียบกัน

4.3.2 การทดลองที่ 2 เปรียบเทียบการโหลดไฟล์ขนาดต่างกัน

การทดสอบประสิทธิภาพของการโหลดไฟล์ ที่เปลี่ยนแปลงไประหว่างการใช่และไม่ใช้แคช และข้อได้เปรียบของแคชที่เก็บไฟล์ไว้ให้โหลดซ้ำได้ทันที ผลการทดลองที่ได้ในตาราง ใช้ผลเฉลี่ยของการทดลองสามครั้ง ในบางการทดลองมีปัญหาทำให้ผลการทดลองผิดพลาดไปหลายวินาที ซึ่งบางกรณีนั้นมากกว่าผลอื่นๆที่ได้จากการทดลองเดียวกัน แต่เนื่องจากทราบสาเหตุที่แน่นอนว่ามาจากปัญหา Endgame จึงตัดผลการทดลองที่ติดปัญหา Endgame ออกในภายหลังเนื่องจากได้พบวิธีแก้ระหว่างการทดลอง

ตอนที่ 1 การโหลดโดยไม่ใช้แคช

ให้เครื่องที่ทำหน้าที่เป็นเครื่องภายในเครือข่าย เรียก Metadata ที่เก็บไว้ออกมาทำงาน เพราะไฟล์ metadata เหล่านี้ไม่ได้ถูกโหลดผ่านพร็อกซี่จึงทำให้โปรแกรม BitComet ติดต่อไปยังเครื่องภายนอกโดยตรง ทำการโหลดทีละไฟล์เพื่อให้ประสิทธิภาพสูงสุด

ผลการทดสอบ

id8:announce43:<http://161.246.5.120/phpBB2/bt/announce.php>10:created by13:BitComet/0.59:

รูปที่ 4.4 URL ของ Tracker ที่ใช้โหลดโดยไม่ผ่านแคช

ตารางที่ 4.5 Client BitComet โหลดไม่ผ่านแคช

File Size	Max DL Rate	Avg DL Rate	Elapse Time
92.48MB	1279 KB/s	679 KB/s	2 min 21 s
200.27 MB	1294 KB/s	637 KB/s	5 min 22 s
386.01 MB	1304 KB/s	740 KB/s	8 min 54 s

สิ่งที่น่าสนใจ มีบางช่วงที่ความเร็วตกลงจากสูงสุดไปยังหยุดนิ่ง เนื่องมาจากการใช้งานเครือข่ายระหว่างแต่ละลง ทำให้มีอุปสรรคการติดต่อต่างๆ ความเร็วการโหลดนั้น จะค่อยๆเพิ่มขึ้นจนถึงความเร็วสูงสุด ดังนั้นในไฟล์ที่มีขนาดเล็ก ความเร็วสูงสุดในการโหลดไฟล์จึงขึ้นไม่ถึงประสิทธิภาพสูงสุดที่ได้ อีกทั้งช่วง 30 MB แรก เป็นช่วงเร่งความเร็ว จึงทำให้ยังโหลดนาน อัตราเฉลี่ยจึงยิ่งสูงขึ้น

ตอนที่ 2 การโหลดโดยใช้แคช

เนื่องจากไฟล์เป็นไฟล์เดียวกัน ให้ทำการลบงานและไฟล์ที่โหลดมาแล้วทั้งสามไฟล์ออกจาก BitComet เพื่อเตรียมเครื่องให้พร้อมที่จะโหลดใหม่ การโหลดผ่านแคช ใช้ Internet Explorer ใต้ URL ของ metadata ไฟล์ตามที่กล่าวไว้เบื้องต้น SquidGuard จะทำการ Redirect ไปที่เว็บเซิร์ฟเวอร์ของแคชเอง ซึ่งตัวแคชจะเริ่มทำงานและโหลดไฟล์ไปก่อนที่เครื่องภายในเครือข่ายจะโหลด ขณะเดียวกันก็โหลด metadata ที่เครื่องภายในจะโหลดมาแก้ไขและ redirect ไฟล์ที่แก้ไขแล้วไปให้แทน และสร้าง Seeder หลอกขึ้นมาในแทรคเกอร์โดยชี้ไอพีแอดเดรสและพอร์ตไปที่ตัวโปรแกรม BitTorrent ของแคช เพื่อให้เครื่องภายในติดต่อกับแทรคเกอร์และโหลดจาก Bittorrent ของแคชเอง

ผลการทดสอบ

Cache

id8:announce43:<http://161.246.5.120/phpBB2/bt/announce.php>10:created by13:BitComet/0.59:

รูปที่ 4.5 URL ของ Tracker ที่แคชใช้ในการโหลด

ตารางที่ 4.6 Cache BitTorrent โหลดพร้อมกับ Client

File Size	Max DL Rate	Avg DL Rate	Elapse Time
92.48MB	1.2 MB/s	1203 KB/s	1 min 20 s
200.27 MB	1.3 MB/s	1207 KB/s	2 min 56 s
386.01 MB	1.3 MB/s	867 KB/s	7 min 25 s

สิ่งที่น่าสนใจ เครื่องแชนเริ่มทำงานโหลดไฟล์ตั้งแต่ช่วงที่เครื่องภายในยังขึ้น Dialog ถามว่าจะเก็บไฟล์ metadata ไว้ที่ไหน ซึ่งทำให้สามารถเริ่มทำการโหลดไปล่วงหน้าได้ เนื่องจากแชนนั้น อยู่ในสถานการณ์เดียวกับเครื่องภายในในการทดสอบแรก ความเร็วในการโหลดของแชนเป็นที่น่าสนใจอย่างมาก ทั้งนี้อาจมีส่วนมาจากประสิทธิภาพของเครื่อง ความแตกต่างของโปรแกรม BitTorrent และการใช้งานเครือข่าย แต่ในไฟล์ขนาดใหญ่ ก็มีช่วงที่เกิดความเร็วตกเช่นกัน

Client BitComet

d8:announce43:<http://161.246.5.224/phpBB2/bt/announce.php>10:created by13:BitComet/0.59

รูปที่ 4.6 URL ของ Tracker ที่ใช้โหลดผ่านแชน

ตารางที่ 4.7 Client BitComet โหลดพร้อมกับแชน

File Size	Max DL Rate	Avg DL Rate	Elastice Time
92.48MB	1407 KB/s	1155 KB/s	1 min 22 s
200.27 MB	1340 KB/s	1159 KB/s	2 min 57 s
386.01 MB	1887 KB/s	847 KB/s	7 min 27 s

สิ่งที่น่าสนใจ โปรแกรม BitComet นั้นแสดงผลว่าติดต่อกับเครื่องแชนอย่างเดียวน และ URL ของแทรคเกอร์ชี้ไปที่เครื่องแชนตามที่ได้ออกแบบไว้ การติดต่อในช่วงแรกเริ่มต้นช้ากว่าการติดต่อไม่ผ่านแชน ทั้งนี้มาจากการทำ Handshake กับไคลเอนต์ที่ต่างชนิดกัน การติดต่อกับฐานข้อมูลของแทรคเกอร์ภายในที่เพิ่งทำการอัปเดต เมื่อเริ่มโหลดไฟล์ Metadata การที่ตัวแชนยังไม่มีข้อมูลที่จะส่งให้เครื่องภายใน และการใช้งานเครือข่ายและทรัพยากรของเครื่องแชนที่ต้องรับข้อมูลจากเครื่องภายนอก และส่งข้อมูลให้เครื่องนี้ไปพร้อมๆกัน แต่ในภายหลัง เมื่อเครื่องแชนทำการโหลดเสร็จสิ้น ความเร็วในการโหลดก็กลับมาเร็วขึ้นมาก ซึ่งความเร็วสูงสุดในการโหลดนั้น สูงกว่าไม่ผ่านแชน

ตอนที่ 3 การโหลดไฟล์เดิมซ้ำผ่านแชน

ทำการลบงานและไฟล์ที่โหลดมาแล้วออกจาก BitComet ของเครื่องภายใน แล้วโหลด metadata ไฟล์อีกครั้งเช่นเดียวกับในตอนที่ 2 แต่ที่จะต่างกันก็คือ ตัวแชนจะตรวจสอบว่างานนี้มีอยู่แล้วหรือไม่ โดยตรวจ Info_Hash ของไฟล์ Metadata เทียบกับฐานข้อมูลของแทรคเกอร์ ซึ่งถ้าพบว่ามียู่แล้ว แชนจะไม่แก้ไขฐานข้อมูล เพื่อสร้าง seeder ปลอมอีก และงานโหลดนั้นยังทำงานอยู่ใน Bittorrent ของแชน

ผลการทดสอบ

d8:announce43:<http://161.246.5.224/phpBB2/bt/announce.php>l0:created by13:BitComet/0.59

รูปที่ 4.7 URL ของ Tracker ที่ใช้โหลดผ่านแคช

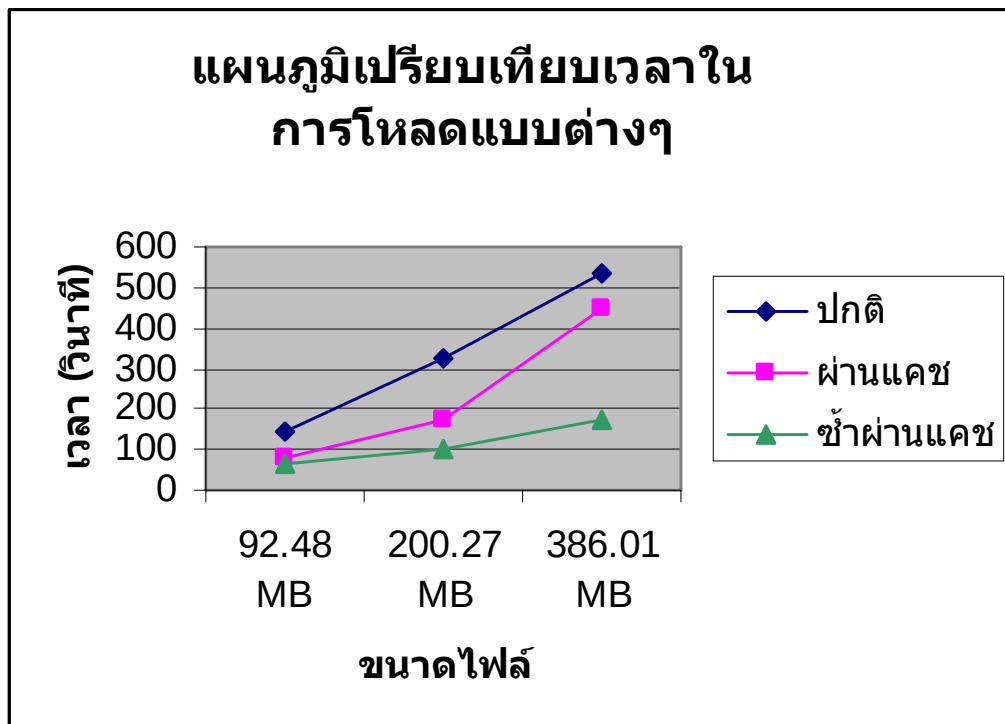
ตารางที่ 4.8 Client BitComet โหลดไฟล์จากแคช

File Size	Max DL Rate	Avg DL Rate	Elapse Time
92.48MB	2715 KB/s	1504 KB/s	1 min 3 s
200.27 MB	2937 KB/s	1973 KB/s	1 min 44 s
386.01 MB	3027 KB/s	2286 KB/s	2 min 53 s

สิ่งที่น่าสนใจ ตัวแคชทำงานตามที่ออกแบบไว้ คือไม่ทำการสร้าง Peer ใหม่ในฐานข้อมูลของแทรคเกอร์ภายในและไม่เรียกงานเข้าซ้อนขึ้นมาทำ ความเร็วในการโหลดจากแคชนั้นเป็นไปได้เร็วกว่าการโหลดโดยตรง หรือเริ่มโหลดพร้อมแคชมาก

จากตารางที่ 4.6 และ 4.9 จะเห็นความแตกต่างของความเร็วในการโหลดได้ชัดเจน ในกรณีที่มีระบบแคช และมีไฟล์อยู่แล้ว การโหลดไฟล์จะทำได้อย่างรวดเร็วมาก เนื่องจากไฟล์นั้นอยู่บนเครื่องในเครือข่าย ทำให้สามารถส่งไฟล์ได้โดยตรง ขณะที่การทำงานแบบปกติ ที่ไม่มีระบบแคช จะต้องโหลดจากภายนอกทุกครั้ง แม้ว่าไฟล์จะซ้ำกันก็ตาม ส่งผลให้ความแตกต่างของทั้งสองตาราง มีค่าอย่างมาก

เมื่อเปรียบเทียบกรณีต่างๆ เฉพาะการทำงานของเครื่องภายในจะพบว่า ผลกระทบที่เกิดจากการนำระบบแคชมาใช้ มีผลเล็กน้อยมาก ซึ่งมาจากการติดต่อทั้งโหลดจากเครื่องภายนอก และการส่งให้เครื่องภายใน ทำให้การโหลดซ้ำในการโหลดพร้อมกับเครื่องภายใน แต่การโหลดซ้ำนั้นแทบจะไม่ส่งผลใดๆเลย สำหรับวิธีลดปัญหาการโหลดซ้ำในการโหลดพร้อมกันกับเครื่องภายในสามารถทำได้โดยเพิ่มจำนวนเครื่องที่เป็นแคช เพื่อลดภาระในการโหลดงานของเครื่องแคช



รูปที่ 4.8 แผนภูมิเปรียบเทียบเวลาการโหลดของเครื่องภายในกรณีต่างๆ

4.4 ปัญหาที่พบระหว่างการทดลองและแนวทางแก้ไข

ปัญหา DHT Function ที่ทำให้ Peer สามารถค้นพบ Peer ตัวอื่นได้โดยไม่ต้องพึ่งแทรคเกอร์ เนื่องจากสภาพการทดสอบแตกต่างจากสภาพจริงคือตำแหน่งการวางเครื่องในเครือข่ายในสภาพการทำงานจริง จะมี Firewall และ Gateway อยู่ ซึ่งสามารถหยุดการค้นหา DHT ได้โดยการปิด UDP Packet ที่ส่งออกนอกเครือข่าย ดังนั้นปัญหานี้ในสภาพการทดสอบ แก้ไขโดยการปิด DHT Function ของตัวเครื่องภายนอกออก รวมถึงสร้างไฟล์ Metadata ที่ไม่รองรับ DHT ข้อแตกต่างระหว่าง Metadata ที่รองรับ DHT และไม่รองรับ DHT คือ แบบที่รองรับ DHT จะมีไอพีแอดเดรสของ DHT Nodes พ่วงที่ท้ายไฟล์หลัง Info field ซึ่งสามารถให้ระบบแคช จัดการทิ้งไปได้

ปัญหา Endgame Download Speed ที่ทำให้การโหลดชะงักที่ 99% พบในโปรแกรมตัวที่ใช้เป็นแคช ทำให้ผลการทดสอบช่วงแรกๆ มีความคลาดเคลื่อน สภาพการทดสอบนั้น มีข้อจำกัดก็คือมี Seeder แค่ตัวเดียว ทำให้ปัญหานี้เกิดขึ้นเป็นประจำ เพราะการทำงาน Endgame คือ การชะลอการโหลด แล้วส่ง Request ของชิ้นส่วนสุดท้ายไปยังทุกๆ Peer ที่มีชิ้นชิ้นนี้ แล้วเมื่อได้รับชิ้นส่วนมาจาก Peer ใดๆ ก็จะส่ง Request_Cancel ไปยัง Peer ตัวอื่นๆ การที่มี Seeder ตัวเดียว ทำให้มีโอกาสที่ Seeder จะรอให้ติดต่อขอจาก Peer ตัวอื่นแทน แต่ในสภาพจริง โอกาสที่งานโหลดบางงานจะมี Seeder แค่ตัวเดียวก็เกิดขึ้นได้ จึงต้องแก้ไขที่โค้ดของโปรแกรมให้ทำงาน Endgame เมื่อเลขจุดที่ควรจะเริ่มไปสักรูปแล้วนำมาทดสอบใหม่

สาเหตุที่เลือกใช้ Clients สำหรับโหนด BitTorrent แต่ละเครื่องไม่เหมือนกัน

BitComet เป็นที่โจษขานกันว่า ละเมิดข้อตกลงของผู้พัฒนาโปรแกรมประเภทนี้ ซึ่งก็คือ การละเลย Private Flag ของไฟล์ Torrent และการให้ DHT Trackerless mode ทำงานแม้จะหยุดในเมนูการปรับแต่งไปแล้วจึงไม่เลือกมาใช้เป็นแเคช แต่เนื่องจากความเป็นที่นิยมจึงนำมาใช้เป็นตัวโหนดภายในเครือข่ายและภายนอก เนื่องจากในสถานการณ์จริงเป็นไปได้ยากที่เราจะไปจำกัดเครื่องในเครือข่ายเราไม่ให้ใช้โปรแกรมใดๆ ที่ไม่ต้องการได้

BitTorrent บน Linux เป็นโค้ดโอเพนซอร์ส ซึ่งสามารถปรับแต่งและแก้ไขโค้ดให้ทำงานตามที่ได้ออกแบบไว้ได้ ซึ่งจำเป็นอย่างมากในการทำเป็น Internal Peer เพราะจำเป็นต้องตั้ง Peer ID ที่ตายตัว และแก้ไขปัญหา Endgame Speed ที่อาจพบได้ใน BitTorrent Client บางตัว

4.5 เปรียบเทียบสภาพการทดลองและสภาพการทำงานจริง

เนื่องจากมีความแตกต่างของสภาพที่ใช้ทดลองกับสภาพการทำงานจริง จำเป็นต้องนำผลการทดลองและข้อที่ต้องคำนึงถึงในสภาพที่ต่างกันมาใช้นำไปประยุกต์ เพื่อให้รู้ว่า ในการใช้งานจริง ระบบนี้มีประสิทธิภาพเช่นใด จะพบปัญหาใดได้บ้าง และระบบเก่ากับระบบแเคชจะจัดการกับปัญหาได้อย่างไร

4.5.1 ปัญหาที่ไม่พบในสภาพการทดลอง

การล่มของแทรคเกอร์

ปัญหาการล่มของแทรคเกอร์นั้น พบได้บ่อยมากในสภาพการทำงานจริง ซึ่งในสภาพจริงนั้น ในระบบปัจจุบันมีการใช้งาน DHT Trackerless Torrent ซึ่งใช้ UDP Port ค้นหา Node และ Peer ตัวอื่นๆ แต่บางครั้ง DHT ก็ไม่ได้รับการสนับสนุนในโปรแกรมที่ต่างกันไป เช่น Azureus ใช้ DHT ที่ไม่เหมือนกับ BitComet และทั้งสองโปรแกรมก็เป็นที่นิยมใช้เช่นกัน ส่วนระบบแเคชนั้น ปัญหาการล่มของแทรคเกอร์ภายใน ขึ้นอยู่กับเครื่องที่ทำงานเป็นแทรคเกอร์ ซึ่งหากระบบเครือข่ายภายในไม่ล่ม และไม่มีปัญหาด้านไฟฟ้าหรืออุปกรณ์แทรคเกอร์ภายในก็มีโอกาสล่มน้อยมาก ส่วนการติดต่อกับแทรคเกอร์ภายนอกของ Internal Peer จะเป็นปัญหาที่แก้ไขได้ยาก เนื่องจาก Internal Peer จะถูกปิดการติดต่อ DHT ไว้เพื่อไม่ให้ส่ง Peers Table ไปให้เครื่องในเครือข่าย แต่ทั้งนี้ ระบบแเคชยังสามารถปล่อยไฟล์ที่เครื่อง Internal Peer โหลดมาไว้ได้แม้จะไม่สามารถโหลดต่อได้ขณะที่แทรคเกอร์ภายนอกล่ม ไม่มี Seeder ที่สมบูรณ์

สภาพไม่มีSeeder ที่สมบูรณ์

ปัญหาสภาพไม่มี Seeder ที่สมบูรณ์ในสภาพจริงนั้นเกิดขึ้นบ่อยมาก โดยเฉพาะในไฟล์ขนาดใหญ่ หรือไฟล์ที่ไม่เป็นที่นิยม ซึ่งเครื่องที่ปล่อยไฟล์เป็นเครื่องแรกอาจปล่อยไม่สมบูรณ์ หรือไม่มีเครื่องอื่นที่โหลดโดยสมบูรณ์ก่อนที่เครื่องเครื่องนั้นจะหยุดแจกจ่ายไฟล์ นอกจากนี้กรณีที่เครื่องที่โหลดจนเกือบสมบูรณ์เกิดปัญหา Endgame ก็จะชะงักที่ 99% จึงไม่สามารถทำให้การโหลดเสร็จสิ้นได้ รวมถึงทำให้เครื่องอื่นๆเจอปัญหาเดียวกัน ทางแก้ของการทำงานปกติ คือพยายามเปิดเครื่องไว้จนกว่าไฟล์จะโหลดเสร็จ ซึ่งสิ้นเปลืองพลังงาน ระบบแคชที่ออกแบบมาไม่สามารถแก้ไข ปัญหาที่เครื่องทั่วไปอาจพบเจอได้ แต่ในทางกลับกัน ระบบสามารถทำงานได้ตลอดเวลา (เนื่องจากถือเป็นส่วนหนึ่งของระบบเครือข่าย) ทำให้สามารถโหลดได้ทันทีที่มี Seeder ใหม่เข้ามาแจกจ่ายไฟล์ นอกจากนี้เครื่องภายในเครือข่ายไม่จำเป็นต้องเปิดไว้ตลอดเวลาเพื่อหา Seeder เนื่องจากระบบจะหาและโหลดมารอไว้ให้ ทำให้สามารถประหยัดการใช้งานได้ด้วย และเมื่อไฟล์ในระบบแคชสมบูรณ์ เครื่องภายในเครือข่ายจะไม่พบสภาพไม่มี Seeder สมบูรณ์สำหรับไฟล์ไฟล์ดังกล่าวเลย